



Nome:

Aquiles Luiz Nunes Bastos **Nº04**

Victor Hugo do Carmo de Jesus **Nº31**

Maria Eduarda Felix Inocêncio **Nº21**

Cainan Bastos da Silva **Nº06**

Série: 3ºA

Professor(a): Andreza Quintas

Matéria: PBE

Lorena/ SP

2026

SPRINT 0 – PLANEJAMENTO (DIA 1 – PRIMEIRAS AULAS)

TIME 2

Etapa 1 – Criar equipe

Definir:

PO (Victor)

Responsável por compreender os requisitos do projeto, organizar o backlog, definir prioridades, validar entregas, alinhar decisões com outros grupos, documentar regras de negócio e representar a equipe durante checkpoints.

Atuação no projeto: levantamento de requisitos, organização das tarefas da sprint, acompanhamento da integração com outros times, validação das entregas e revisão da documentação.

Entregas: backlog organizado, checklist da sprint, documentação dos endpoints, regras de negócio documentadas e planejamento da integração entre equipes.

Scrum Master (Aquiles)

Responsável por organizar o trabalho da equipe, controlar o tempo da sprint, garantir o cumprimento dos critérios do projeto, registrar impedimentos, mediar conflitos e conduzir reuniões rápidas (Daily Scrum).

Atuação no projeto: acompanhamento das atividades, organização da Sprint Board, controle do cronograma, identificação de riscos e monitoramento do progresso da equipe.

Entregas: relatório de andamento, registro de impedimentos, evidências de progresso, organização das tarefas e acompanhamento das metas da sprint.

Desenvolvedores Técnicos (Cainan e Maria Eduarda)

Responsáveis pela implementação do código, modelagem técnica, organização das rotas, testes no Postman, padronização das respostas da API e documentação técnica.

Atuação no projeto: desenvolvimento da arquitetura em camadas, criação das camadas Controller, Service e Repository, organização das pastas, testes dos endpoints e documentação da solução.

Entregas: estrutura do projeto organizada, endpoints funcionais, testes executados, documentação técnica e evidências da refatoração.

Etapa 2 – Ler requisitos do módulo

A equipe deve responder:

1. O que nosso módulo faz?

Somos responsáveis por estruturar toda a base arquitetural do Sistema de Gestão de Eventos. Nossa função é organizar o projeto utilizando Arquitetura em Camadas, separando responsabilidades entre Controller, Service e Repository. Essa abordagem reduz a complexidade do código, melhora a manutenção, aumenta a reutilização e facilita futuras integrações. A arquitetura criada por nossa equipe servirá como base para que os demais grupos desenvolvam suas funcionalidades de forma organizada e padronizada.

2. O que precisamos desenvolver?

Precisamos implementar a estrutura completa da Arquitetura em Camadas, organizando as pastas e separando corretamente as responsabilidades do sistema. Será necessário criar Controllers para receber requisições, Services para aplicar regras de negócio e Repositories para acessar os dados. Também devemos demonstrar uma refatoração mostrando o antes e depois da arquitetura, documentar as decisões técnicas tomadas e garantir compatibilidade com os demais módulos.

3. Quem depende de nós?

Os grupos 3, 4, 5, 6 e 7 dependem diretamente da arquitetura criada pelo Time 2. O Grupo 3 utilizará nossa estrutura para implementar DTOs e tratamento global de erros. O Grupo 4 utilizará a arquitetura para aplicar autenticação e segurança. O Grupo 5 dependerá da organização para integrar APIs externas e Webhooks. O Grupo 6 utilizará a estrutura para upload e download de arquivos. Já o Grupo 7 utilizará nossa arquitetura para implementar logs, observabilidade e monitoramento do sistema.

4. De quem dependemos?

Dependemos principalmente do Grupo 1, responsável pelo CRUD de Eventos. A partir da implementação realizada por eles, faremos a refatoração e organização do sistema utilizando Arquitetura em Camadas. Também dependemos do alinhamento com os demais grupos para garantir que todos utilizem os mesmos padrões de integração, respostas JSON e organização do projeto.

Arquitetura em Camadas

Controller: recebe requisições HTTP e retorna respostas.

Service: aplica regras de negócio e validações.

Repository: acessa e manipula os dados.

Estrutura de Pastas

src/

controllers/

services/

repositories/

routes/

models/

middlewares/

app.js

Refatoração

ANTES: Controller concentrava regras de negócio, validações e acesso ao banco.

DEPOIS: Controller → Service → Repository → Banco de Dados.

Fluxo da Aplicação

Cliente → Controller → Service → Repository → Banco de Dados → Resposta

Backlog

Arquitetura em camadas

Fazer 6

- ☐ Controller
- ☐ Service
- ☐ Repository
- ☐ refatoração demonstrada
- ☐ Checklist da Sprint
- ☐ Organização da Sprint Board

+ Adicionar um cartão

Fazendo 3

- ☐ Lista de impedimentos
- ☐ Evidência do progresso
- ☐ organização de pastas

+ Adicionar um cartão

Concluído 3

- ☐ backlog organizado
- ☐ Regras de negócio
- ☐ Relatório de andamento

+ Adicionar um cartão

Requisitos Funcionais (RF)

RF01 – O sistema deve possuir uma camada de Controller para receber e responder às requisições HTTP.

RF02 – O sistema deve possuir uma camada de Service responsável pelas regras de negócio.

RF03 – O sistema deve possuir uma camada de Repository responsável pelo acesso aos dados.

RF04 – O Controller deve encaminhar as requisições para a camada Service.

RF05 – A camada Service deve realizar as validações e processamentos necessários.

RF06 – A camada Service deve utilizar a camada Repository para operações de persistência de dados.

RF07 – O sistema deve permitir operações de cadastro, consulta, atualização e exclusão de eventos por meio da arquitetura em camadas.

RF08 – O sistema deve retornar respostas padronizadas para todas as operações da API.

RF09 – O sistema deve permitir integração com os módulos desenvolvidos pelos demais times.

RF10 – O sistema deve manter a separação entre lógica de negócio e acesso aos dados.

Requisitos Não Funcionais (RNF)

RNF01 – O sistema deve utilizar arquitetura em camadas (Controller, Service e Repository).

RNF02 – O código-fonte deve seguir padrões de organização e boas práticas de desenvolvimento.

RNF03 – A estrutura do projeto deve ser organizada em pastas específicas para cada camada.

RNF04 – O sistema deve ser desenvolvido utilizando Node.js e Express.js.

RNF05 – As respostas da API devem utilizar o formato JSON.

RNF06 – O sistema deve ser versionado utilizando Git e GitHub.

RNF07 – A arquitetura deve permitir manutenção e expansão futura sem grandes alterações na estrutura existente.

RNF08 – O sistema deve possibilitar integração com APIs externas e novos módulos.

RNF09 – O código deve ser documentado para facilitar o entendimento da equipe.

RNF10 – O sistema deve possuir baixo acoplamento entre as camadas, garantindo maior reutilização e manutenção do código.

Evidências

←

T

TIME 2 ▾

4 participantes

🔍

🖼️

📁

✅

🗑️

✎



Cainan Silva 3 min

Opa pessoal

3 min

Olá

Vou mandar a função de cada integrante e o nosso documento

<https://docs.google.com/document/d/18lHBgbxHuNVITv3TbUihmZWOD7CkCzczDOOucRc1x0/edit?usp=sharing>



Nome:

Aquiles Luiz Nunes Bastos Nº04

Victor Hugo do Carmo de Jesus Nº31

Maria Eduarda Farias Inocência Nº21

Cainan Bastos da Silva Nº06

ARQUITETURA EM CAMADAS



PO (Victor)

←

D

DEV'S EVENTHUB SOLUTIONS ▾

9 participantes

🔍

🖼️

📁

✅

🗑️

✎

semana que vem que a gente reúne pra programar

Francisco Gabriel 10:13

F

Frontend

HTML, CSS e JavaScript.

Backend

Node.js com Express.js.

Banco de Dados

MySQL.

Formato de Dados

JSON.

Autenticação

Token Bearer.

Neil Lopes 10:27

boa

eu sei como que hospeda negocio

sem pagar

com bd front end e backend

Francisco Gabriel 10:28

F

render

Neil Lopes 10:28

+

O histórico está ativado

A

🗨️

📄

📶

🎤

⏮

⏭

Arquitetura em camadas

VC

CS

AB

ME

Compartilhar

Fazer7

Controller

Service

Repository

organização de pastas

refatoração demonstrada

Checklist da Sprint

Organização da Sprint Board

+ Adicionar um cartão

Fazendo4

Relatório de andamento

Lista de impedimentos

Evidência do progresso

Regras de negócio

+ Adicionar um cartão

Concluído1

backlog organizado

+ Adicionar um cartão

+ Adicionar outra lista

